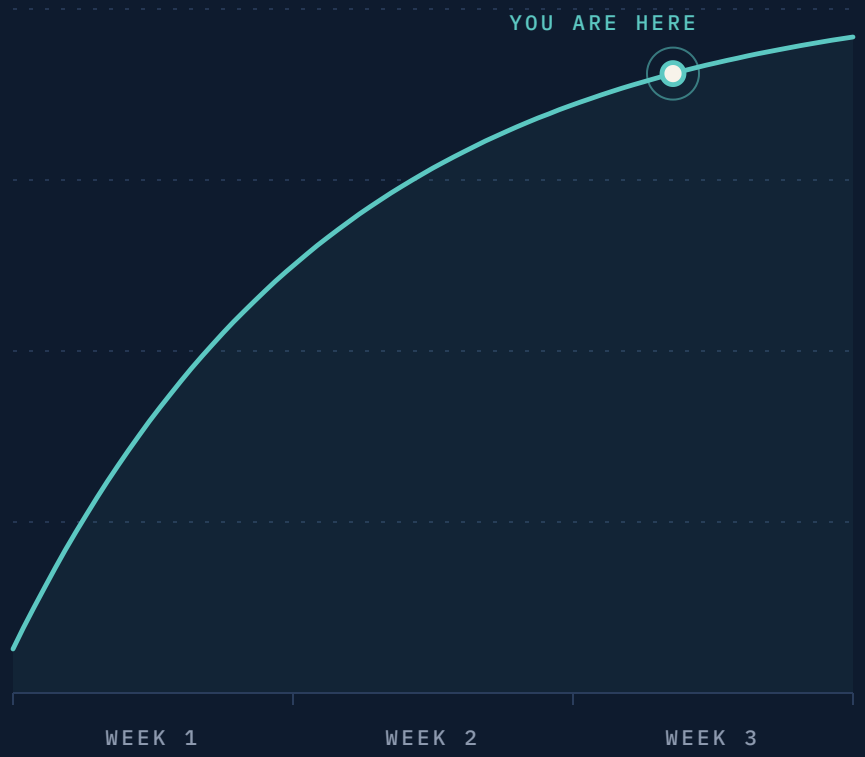
WEEK 3 · DAY 2 · OVERVIEW

SQL Joins and Time Series



Created by **Jeff Lenamon**

ABOUT 50 MIN WITH EXERCISES / 8 SECTIONS / 4 EXERCISES

This course is educational content created in a personal capacity. Nothing here is investment advice or a recommendation to buy or sell any security. All portfolio data is synthetic.

The Desk Lead's Note

September's blotter is 374 trades and 117,473,749.35 dollars of principal. Tell me how much traded in each grade. Then tell me which benchmark bonds we do not hold, and how much benchmark weight they carry. Last, give me the 10-year's daily changes and month-end levels straight from the database, and show me they match our own functions. Every total has to tie back to the blotter.

ABOUT 50 MIN WITH EXERCISES / 8 SECTIONS / 4 EXERCISES

Your Route Through Day 2

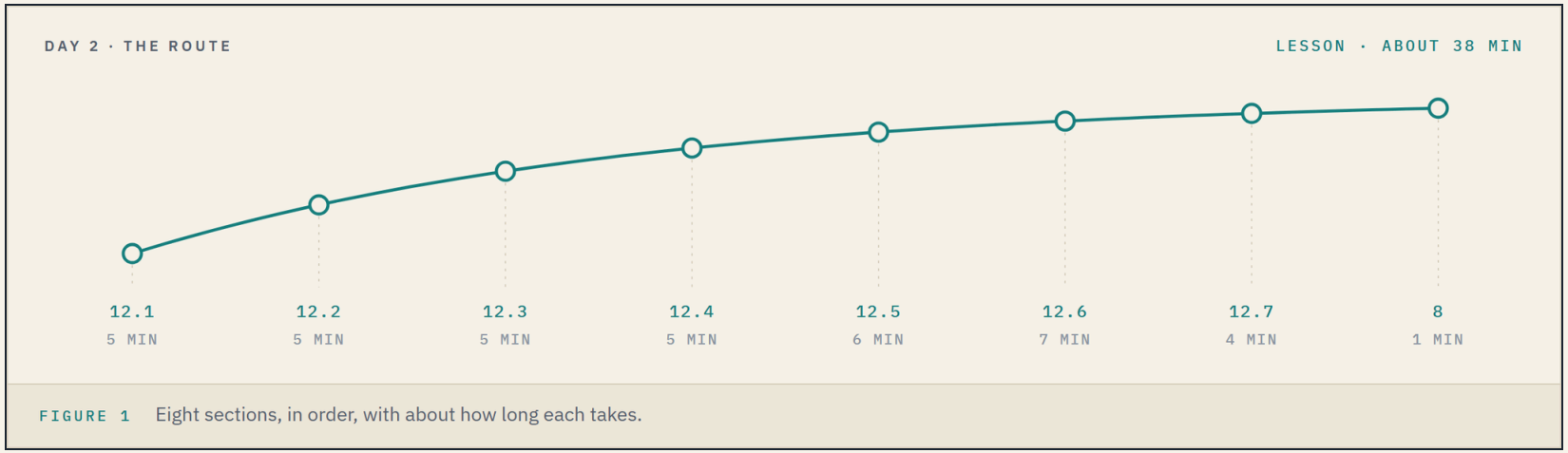


FIGURE 1 Eight sections, in order, with about how long each takes.

	SECTION	ABOUT
12.1	Seven Tables and One Control Figure	5 min
12.2	JOIN: Every Trade Finds Its Grade	5 min
12.3	LEFT JOIN: The Benchmark Bonds the Book Does Not Hold	5 min
12.4	The Duplicate Trap	5 min

	SECTION	ABOUT
12.5	Dates as Text: **`BETWEEN`** and **`substr`**	6 min
12.6	Window Functions: **`LAG`** and **`ROW_NUMBER`**	7 min
12.7	Two Tests and the History of **`queries.sql`**	4 min
8	Bring It Home	1 min

Time. About 38 minutes for the lesson, then about 12 for the four exercises.

12.1 Seven Tables and One Control Figure

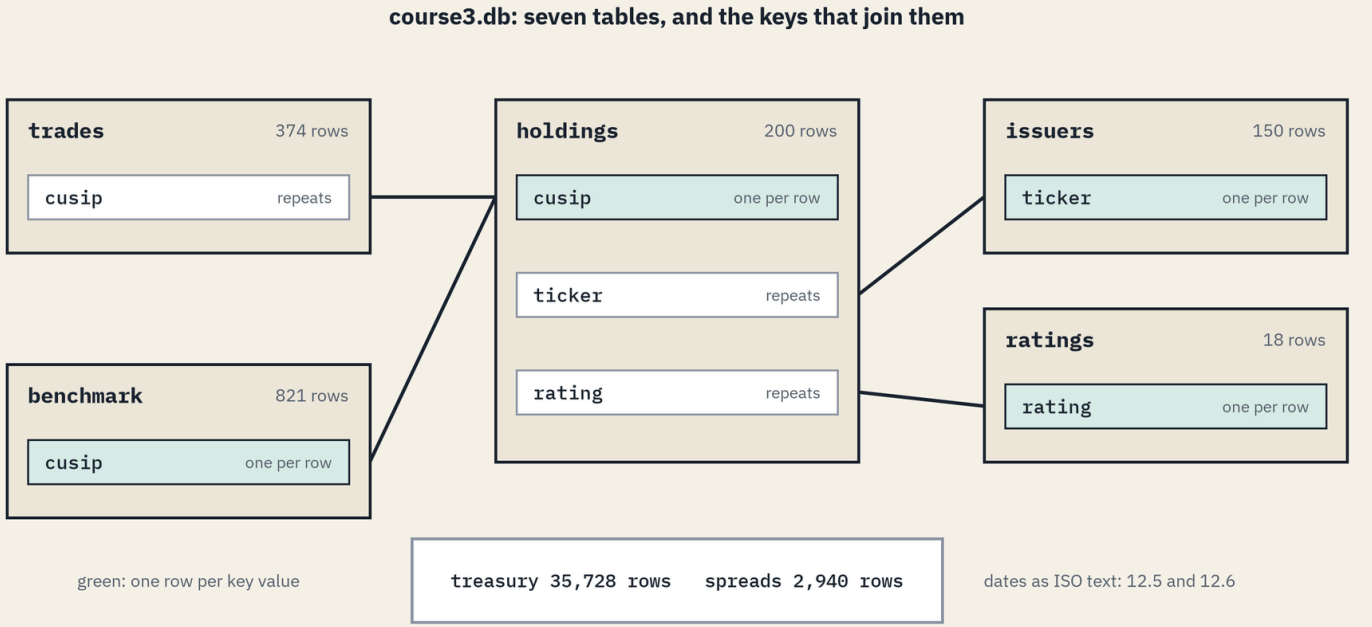


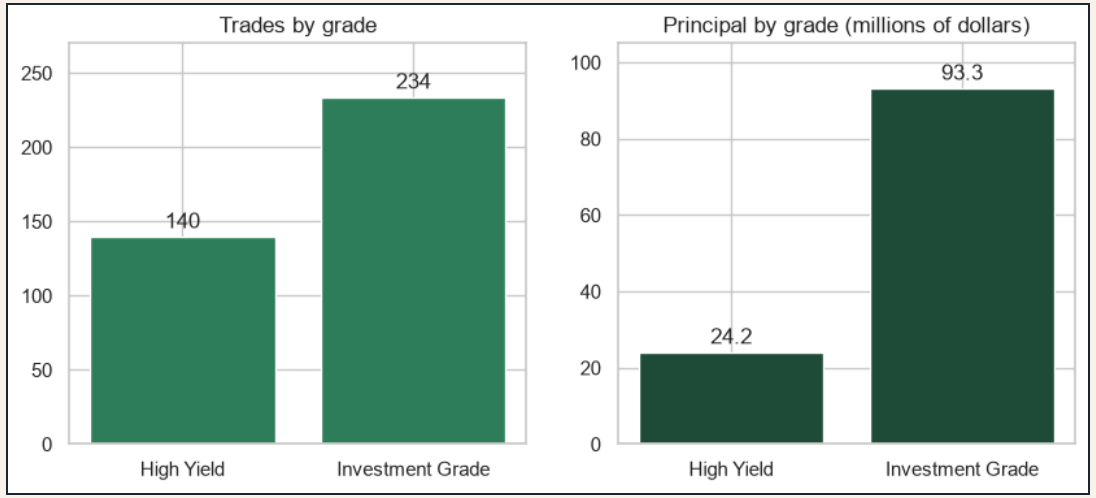
FIGURE 2 The seven tables of course3.db with their row counts, joined on cusip, ticker, and rating keys.

THE CONTROL FIGURE

```
374 trades, 2026-09-01 to 2026-09-30
principal 117,473,749.35 dollars
```

Seven tables in one file, and one control figure every trade join must keep: 374 trades, 117,473,749.35 dollars.

12.2 JOIN: Every Trade Finds Its Grade



THE JOIN

```
by_grade = md.run_query(db_path, '''
    SELECT r.grade, COUNT(*) AS trades, SUM(t.principal) AS principal
    FROM trades AS t
    JOIN holdings AS h ON t.cusip = h.cusip
    JOIN ratings AS r ON h.rating = r.rating
    GROUP BY r.grade
    ORDER BY r.grade''')
by_grade.style.hide(axis='index').format({'principal': '{:,.2f}'})
```

EXCEL SIDE BY SIDE

JOB	EXCEL	RESULT
Does a key repeat?	`=COUNTIF($B$2:$B$201,B2)` filled down	145 rows above 1, largest 6; on cusip, largest 1
Each trade's bond, by cusip	`=XLOOKUP(F2,$A$2:$A$201,$C$2:$C$201)` filled down, the sector	374 trades, no `#N/A`

Two joins carried all 374 trades to a grade, and the blotter's 117,473,749.35 dollars tie to the cent.

12.3 LEFT JOIN: The Benchmark Bonds the Book Does Not Hold

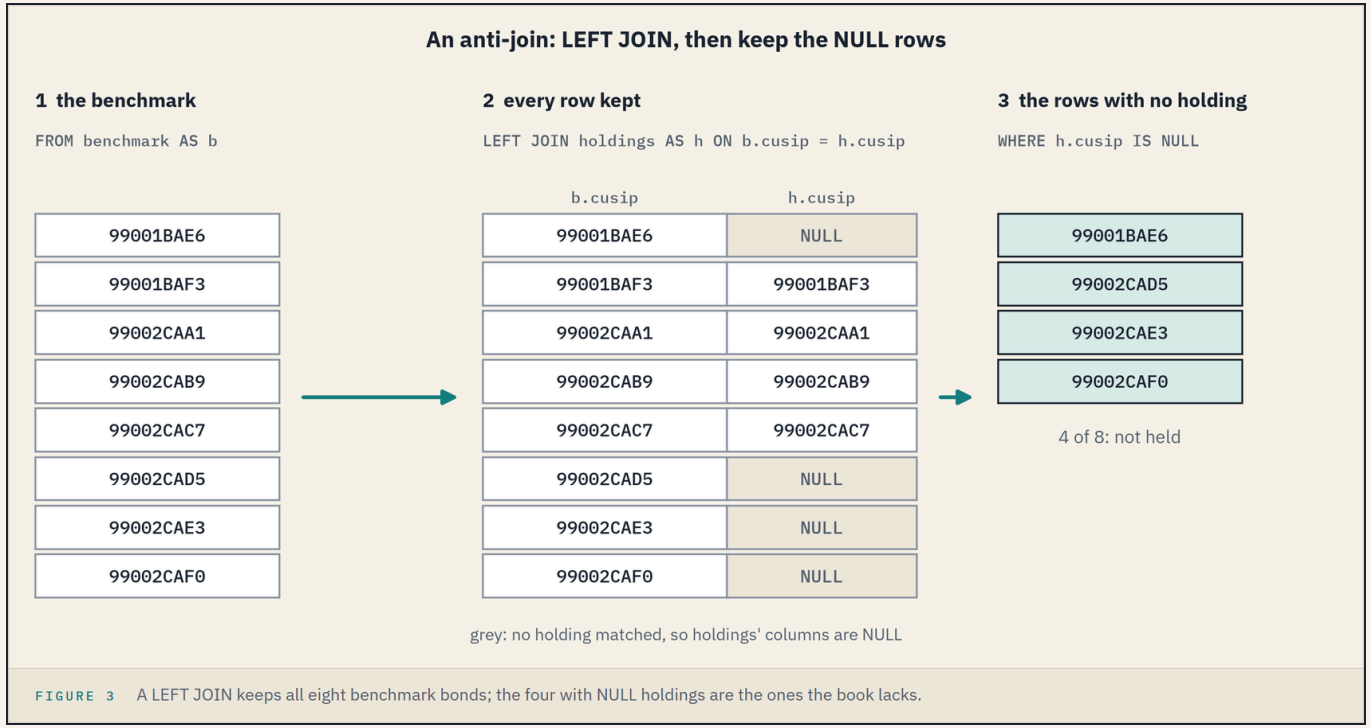


FIGURE 3 A LEFT JOIN keeps all eight benchmark bonds; the four with NULL holdings are the ones the book lacks.

THE ANTI-JOIN

```
not_held = md.run_query(db_path, '''
    SELECT b.cusip, b.ticker, b.sector, b.weight_pct
    FROM benchmark AS b
    LEFT JOIN holdings AS h ON b.cusip = h.cusip
    WHERE h.cusip IS NULL''')
```

```
621 of 821 benchmark bonds are not in the book
```

621 benchmark bonds are not in the book and carry 75.44 percent of the weight: a LEFT JOIN finds what an inner join throws away.

12.4 The Duplicate Trap

WHICH KEY REPEATS

```
    SELECT ticker, COUNT(*) AS bonds
    FROM benchmark
    GROUP BY ticker
    HAVING COUNT(*) > 1
    ORDER BY bonds DESC, ticker''')
```

```
150 tickers repeat, up to 8 bonds each
```

THE FIX: JOIN ON CUSIP

```
200 rows, market value unchanged
```

PANDAS REFUSES

```
MergeError: Merge keys are not unique in right dataset; not a many-to-one merge

Duplicates in right:
 ticker
  QVNE
  QVNE
  QVNE
  QVNE
  QVNE ...
```

Count before, count after, compare a total: a key that repeats multiplies rows and dollars with no error, and pandas' `validate=` is the only side that stops.

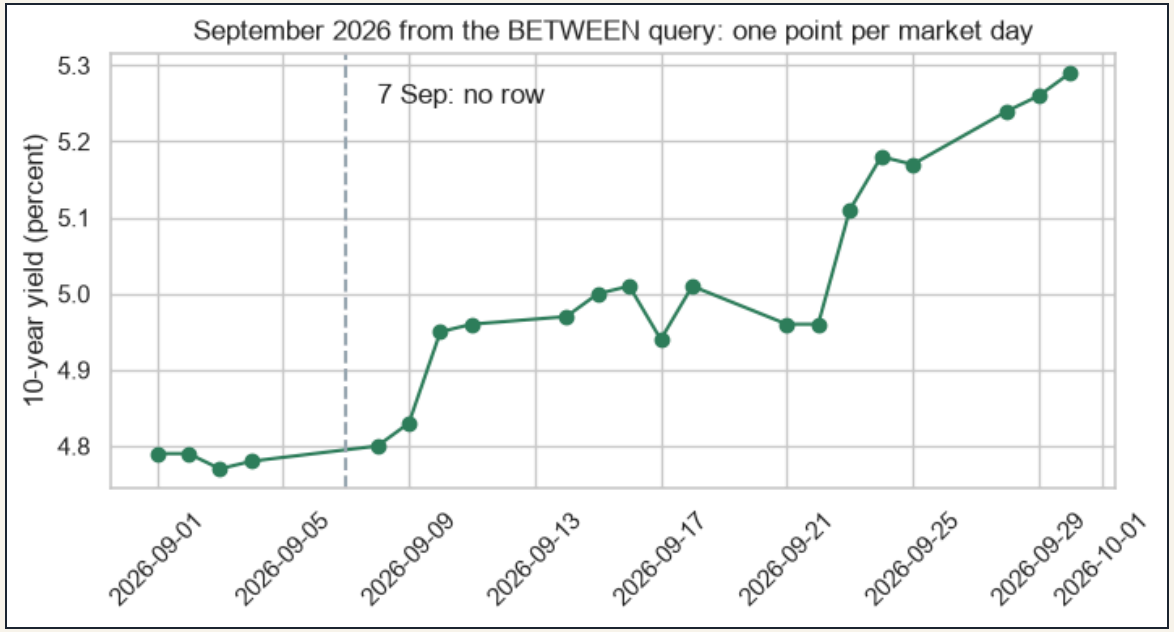
12.5 Dates as Text: BETWEEN and substr

SEPTEMBER WITH BETWEEN

```
september = md.run_query(db_path, '''
    SELECT date, yield_pct
    FROM treasury
    WHERE tenor = ? AND date BETWEEN ? AND ?
    ORDER BY date''', ('10 Yr', '2026-09-01', '2026-09-30'))
```

EACH MONTH OF 2026

MONTH	LOW_PCT	HIGH_PCT	AVERAGE_PCT
2026-01	4.15	4.30	4.213
2026-02	3.97	4.29	4.126
2026-03	4.05	4.44	4.246
2026-04	4.26	4.42	4.319



ISO text compares in date order only at full width; `BETWEEN` keeps both ends, and the first seven characters are the month.

12.6 Window Functions: LAG and ROW_NUMBER



FIGURE 6 LAG carries the prior row's yield down; ROW_NUMBER starts again at 1 in each month.

EXCEL SIDE BY SIDE

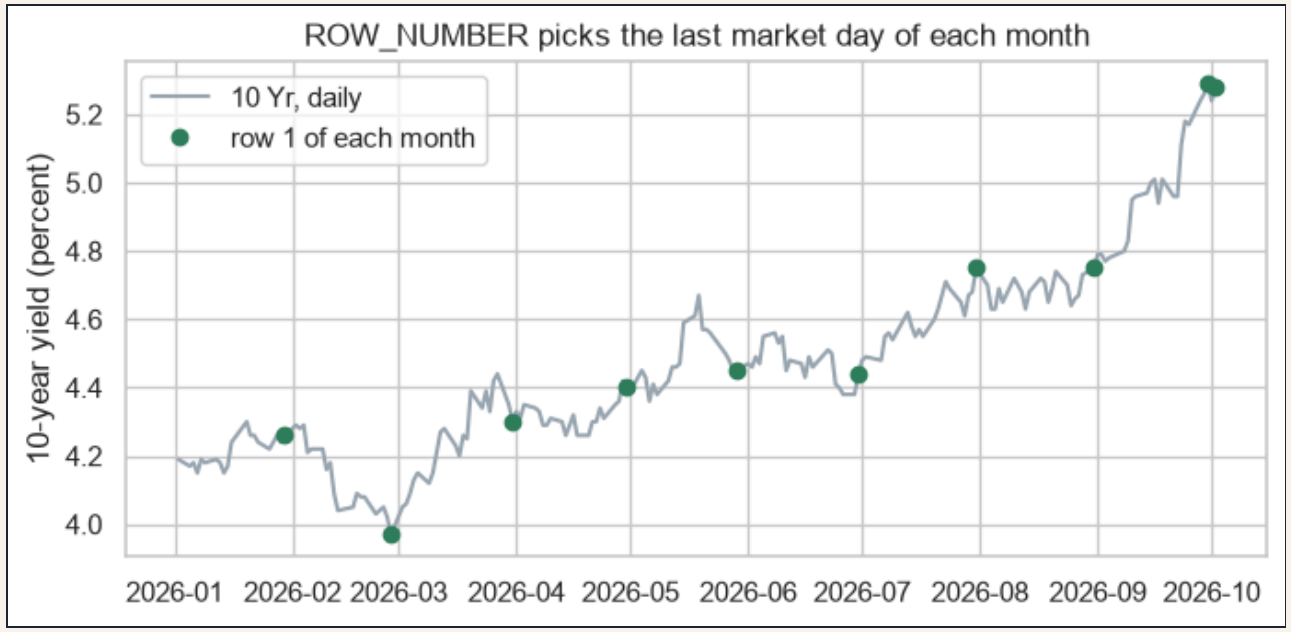
`LAG` is the formula you would type anyway: `=(B3-B2)*100`, filled down.

PROVED AGAINST CHANGE_BP

```
LAG equals change_bp on all 2,940 rows
```

Each change is from the previous row, whatever the calendar gap: notebook 06's rule.

Month Ends With ROW_NUMBER



THE LAST THREE MONTH ENDS

```
date  yield_pct
139  2026-08-31       4.75
140  2026-09-30       5.29
141  2026-10-02       5.28
```

PROVED AGAINST LAST_PER_PERIOD

```
ROW_NUMBER equals last_per_period on all 142 months
```

The database gives the 10-year's daily changes and month-end levels, equal to `change_bp` and `last_per_period` on every row: compute the window first, then filter.

12.7 Two Tests and the History of queries.sql

THE TWO TESTS

```
def test_trades_join_holdings_on_cusip_keeps_rows(tmp_path):
    assert len(joined) == 374
    assert round(joined["principal"].sum(), 2) == 117_473_749.35
def test_month_end_sql_matches_last_per_period(tmp_path):
    assert len(month_end_sql) == 142
```

WHAT CHANGED

```
!git -C "{work}" status --short
```

```
 M test_marketdata.py
?? treasury_long.csv
```

THE DIFF

```
!git -C "{work}" diff --stat
```

```
queries.sql        | 61 ++++++
```

Bring It Home

IN A POWERSHELL TERMINAL

```
# the work folder: use the name your setup cell printed
$work = "$env:TEMP\pfi_course3_12_k4w7q2xr"
Set-Location "$HOME\projects\my_bondmath"
Copy-Item "$work\marketdata.py", "$work\test_marketdata.py", "$work\queries.sql" .
Copy-Item "$work\trade_blotter.csv" .
python -m pytest -q test_marketdata.py
git status --short
git add marketdata.py test_marketdata.py queries.sql
git commit -m "Notebook 12: tests of joins and month ends, and the day's queries.sql"
```

pytest shows 48 passed.

You Can Now

- Check a key with `COUNT(DISTINCT ...)` before a join, and join tables with `JOIN ... ON` and table names such as `h`.
- Find rows with no match with `LEFT JOIN ... WHERE ... IS NULL`, and with `indicator=True` in pandas.
- Count before and after every join, tie a control figure, and stop a repeated key with `validate=`.
- Select dates stored as ISO text with `BETWEEN`, and group them by month with `substr`.
- Use `LAG` and `ROW_NUMBER` for daily changes and month ends, and filter after the window.
- Ask Git which commits changed one file.

NEXT
WEEK 3 · DAY 3

Working With an AI Assistant

An assistant's "this passes your test" checked with tests you run: three pandas answers, a chart, a DV01, what may go into the chat, a package that does not exist, JSON replies read by parse_json_reply, and a patch reviewed with Git.